

CSES Dynamic Programming

A Problem-Solving Companion to the CSES Problem Set

Maruri Saikiran

August 10, 2026

Contents

1	Dynamic Programming	1
1.1	The Three Questions Behind Every DP	1
1.2	Dice Combinations	3
1.3	Minimizing Coins	4
1.4	Coin Combinations I	5
1.5	Coin Combinations II	7
1.6	Removing Digits	8
1.7	Grid Paths	10
1.8	Book Shop	11
1.9	Array Description	13
1.10	Counting Towers	14
1.11	Edit Distance	16
1.12	Rectangle Cutting	17
1.13	Money Sums	19
1.14	Removal Game	20
1.15	Two Sets II	22
1.16	Increasing Subsequence	23
1.17	Projects	25
1.18	Elevator Rides	26
1.19	Counting Tilings	28
1.20	Counting Numbers	30

Chapter 1

Dynamic Programming

Dynamic programming is brute force made efficient by never solving the same subproblem twice. Every problem in the CSES DP section is the same three questions: what is the smallest piece of state that describes a subproblem, how does a subproblem's answer combine its smaller subproblems, and in what order must they be filled so each dependency is ready. Answer those three and the code is mechanical.

1.1 The Three Questions Behind Every DP

A dynamic program is a recurrence plus memory. The art is entirely in choosing the *state*; once the state and its transition are right, the implementation is a loop.

State, Transition, Order

For every problem ask:

1. **State.** What parameters uniquely identify a subproblem? (a prefix length, a remaining sum, a cell (i, j) , a bitmask of used items.) The number of states times the cost per state is the complexity.
2. **Transition.** How is $dp[\text{state}]$ built from smaller states? This is the recurrence – usually “try every last choice.”
3. **Base case & order.** Which states are known outright, and in what order must states be computed so every dependency is already done? (Iterative bottom-up, or memoised top-down.)

How to Recognise the DP Type

CSES DP Families

- “Count the number of ways” / “minimum coins” – one-dimensional DP over a target value: Dice Combinations, Minimizing Coins, Coin Combinations I/II, Removing Digits, Money Sums. The key subtlety is *ordered vs unordered* counting (permutations vs combinations), which is decided by loop order.
- **Grid / two-sequence DP** – state is a cell (i, j) : Grid Paths, Edit Distance, Counting Tilings.
- **Knapsack-style** – items with weights/values, state is (prefix, capacity): Book Shop, Money Sums, Two Sets II.
- **Interval / game DP** – state is a subarray $[l, r]$: Removal Game, Rectangle Cutting.
- **Bitmask DP** – state includes a subset: Elevator Rides, Counting Tilings (profile), Two Sets.

- **Digit DP** – count numbers with a property up to N digit by digit: Counting Numbers.
- **Classic sequence DP** – Increasing Subsequence (patience sorting), Array Description, Projects (weighted interval scheduling), Counting Towers.

Each section states the problem, identifies the state and transition, gives the recurrence and its order, a hand-traced dry run, complete compilable C++ (using `long long` and the CSES modulus $10^9 + 7$ where the problem counts things), and a complexity analysis.

1.2 Dice Combinations

Problem Statement. Count the number of ways to construct the sum n by throwing a die one or more times, where each throw yields 1 to 6 and different *orders* count as different ways. Output the count modulo $10^9 + 7$.

Pattern Recognition

“Number of ways” + “order matters” is a one-dimensional counting DP over the target sum. Let $dp[x]$ be the number of ordered sequences summing to x ; the last throw was some value $1 \dots 6$, so $dp[x]$ sums the ways to have reached $x - 1, \dots, x - 6$ before that final throw.

State and transition

Define $dp[x]$ as the number of ways to obtain sum x . The final die throw contributes a value $k \in \{1, \dots, 6\}$, and everything before it must sum to $x - k$. Since every different final value (and every different earlier sequence) is a distinct ordered way, we simply add:

$$dp[x] = \sum_{k=1}^6 dp[x - k], \quad dp[0] = 1.$$

The base $dp[0] = 1$ counts the single empty sequence.

Ordered Counting: Sum Over the Last Move

Because order matters, we classify sequences by their *last* throw. Summing over all six possible last throws counts every ordered sequence exactly once, with no double counting – each sequence has a unique final element.

Algorithm

1. $dp[0] = 1$.
2. For $x = 1 \dots n$: $dp[x] = \sum_{k=1}^6 dp[x - k]$ (skipping $x - k < 0$), taken modulo $10^9 + 7$.
3. Answer is $dp[n]$.

Dry run

Dry Run. $n = 3$.

- $dp[0] = 1$.
- $dp[1] = dp[0] = 1$ (just “1”).
- $dp[2] = dp[1] + dp[0] = 2$ (“1+1”, “2”).
- $dp[3] = dp[2] + dp[1] + dp[0] = 2 + 1 + 1 = 4$ (“1+1+1”, “1+2”, “2+1”, “3”).

C++ implementation

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 const int MOD = 1e9 + 7;
4
```

```

5 int main() {
6     int n;
7     cin >> n;
8     vector<long long> dp(n + 1, 0);
9     dp[0] = 1;
10    for (int x = 1; x <= n; x++)
11        for (int k = 1; k <= 6; k++)
12            if (x - k >= 0) dp[x] = (dp[x] + dp[x - k]) % MOD;
13    cout << dp[n] << "\n";
14    return 0;
15 }

```

Complexity Analysis

Time Complexity. $O(6n) = O(n)$ – each of n states sums six predecessors. **Space Complexity.** $O(n)$ for the DP array (reducible to $O(1)$ by keeping only the last six values).

Key Takeaway

The archetypal ordered-counting DP: *sum over the last move*. Every “count sequences where order matters” problem in this chapter is a variation on this single recurrence – the only thing that changes is the set of allowed last moves.

1.3 Minimizing Coins

Problem Statement. Given n coin values and a target sum x , find the *minimum* number of coins needed to make exactly x (each coin may be used any number of times), or report -1 if it is impossible.

Pattern Recognition

“Minimum coins to make a sum,” *unlimited supply*, is the **unbounded knapsack** minimisation. Let $dp[s]$ be the fewest coins summing to s ; the last coin used is some value c , so $dp[s] = 1 + \min_c dp[s - c]$.

State and transition

$dp[s]$ = minimum coins to make sum s , with $dp[0] = 0$ and all others initialised to “infinity” (unreachable). For each s , try every coin $c \leq s$ as the last coin: if $s - c$ is reachable, s can be reached with one more coin. Take the best:

$$dp[s] = \min_{c: c \leq s} (dp[s - c] + 1).$$

Minimisation Instead of Counting

This is Dice Combinations with $\min + 1$ replacing summation: instead of counting all ways to make the last move, we keep only the cheapest. Coins may repeat, so every coin is available at every sum – that is what makes it *unbounded*.

Algorithm

1. $dp[0] = 0$; $dp[s] = \infty$ for $s \geq 1$.

2. For $s = 1 \dots x$ and each coin $c \leq s$: if $dp[s - c]$ is finite, $dp[s] = \min(dp[s], dp[s - c] + 1)$.
3. Answer is $dp[x]$, or -1 if it is still ∞ .

Dry run

Dry Run. Coins $\{1, 5, 7\}$, $x = 11$.

- $dp[5] = 1$ (one 5), $dp[7] = 1$, $dp[10] = 2$ (5 + 5).
- $dp[11]$: via coin 1 from $dp[10] = 2 \rightarrow 3$; via coin 5 from $dp[6]$ (= 2, as 5+1) $\rightarrow 3$; via coin 7 from $dp[4]$ (= 4) $\rightarrow 5$. Minimum = 3.
- Answer 3 (e.g. 5 + 5 + 1).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, x;
6     cin >> n >> x;
7     vector<int> coins(n);
8     for (int& c : coins) cin >> c;
9
10    const int INF = 1e9;
11    vector<int> dp(x + 1, INF);
12    dp[0] = 0;
13    for (int s = 1; s <= x; s++)
14        for (int c : coins)
15            if (c <= s && dp[s - c] != INF)
16                dp[s] = min(dp[s], dp[s - c] + 1);
17
18    cout << (dp[x] == INF ? -1 : dp[x]) << "\n";
19    return 0;
20 }
```

Complexity Analysis

Time Complexity. $O(nx)$ – each of x sums tries all n coins. **Space Complexity.** $O(x)$ for the DP array.

Key Takeaway

Swap “sum of ways” for “min of ways +1” and a counting DP becomes an optimisation DP. The unbounded (coins reusable) structure is why every coin is tried at every sum, distinguishing this from the 0/1 knapsack of Book Shop (Section 1.8).

1.4 Coin Combinations I

Problem Statement. Given n coins and a target x , count the number of *ordered* ways to make x (a coin may be used many times, and sequences differing only in order are counted separately). Output modulo $10^9 + 7$.

Pattern Recognition

“Count ways, order matters” with reusable coins is Dice Combinations generalised to an arbitrary coin set. Let $dp[s]$ count ordered sequences summing to s ; the last coin is any c , so $dp[s] = \sum_c dp[s - c]$.

State, transition, and loop order

$dp[s]$ counts ordered sequences summing to s , $dp[0] = 1$. The last coin used has some value c ; the prefix before it sums to $s - c$:

$$dp[s] = \sum_{c: c \leq s} dp[s - c].$$

The crucial detail is the **loop order**: the *sum* s runs on the outside and coins on the inside. This way every coin is considered as a possible *last* move for each sum, so (2 then 3) and (3 then 2) are both counted – exactly what “order matters” requires.

Sum-Outer Loop Counts Permutations

Putting the sum on the outer loop lets each sum choose *any* coin as its last step, generating all orderings. Reversing the loops (coins outer) would count each multiset once – the distinction between this problem and Coin Combinations II (Section 1.5).

Algorithm

1. $dp[0] = 1$.
2. For $s = 1 \dots x$ (outer): for each coin $c \leq s$ (inner), $dp[s] += dp[s - c]$, modulo $10^9 + 7$.
3. Answer $dp[x]$.

Dry run

Dry Run. Coins $\{2, 3, 5\}$, $x = 9$; partial values.

- $dp[0] = 1$, $dp[2] = 1$, $dp[3] = 1$, $dp[5] = dp[3] + dp[2] + dp[0] = 3$ (2+3, 3+2, 5).
- Building up, $dp[9] = 8$ – all ordered sequences of $\{2, 3, 5\}$ summing to 9.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int MOD = 1e9 + 7;
4
5 int main() {
6     int n, x;
7     cin >> n >> x;
8     vector<int> coins(n);
9     for (int& c : coins) cin >> c;
10
11     vector<long long> dp(x + 1, 0);
12     dp[0] = 1;
13     for (int s = 1; s <= x; s++) // sum outer -> permutations
14         for (int c : coins)

```

```

15         if (c <= s) dp[s] = (dp[s] + dp[s - c]) % MOD;
16
17     cout << dp[x] << "\n";
18     return 0;
19 }
```

Complexity Analysis

Time Complexity. $O(nx)$. **Space Complexity.** $O(x)$.

Key Takeaway

Ordered coin counting = sum-outer, coin-inner loops. Memorise this loop order as the switch that toggles between counting permutations (here) and combinations (next section). Getting it backward is the single most common DP counting bug.

1.5 Coin Combinations II

Problem Statement. Given n coins and a target x , count the number of *distinct* ways to make x where the *order does not matter* (each multiset of coins is counted once). Output modulo $10^9 + 7$.

Pattern Recognition

“Count ways, order does not matter” is the combination count. The trick that removes duplicate orderings is to fix a global order on the coins and only ever extend a combination with coins “at or after” the current one – realised by putting the **coin loop on the outside**.

Why coin-outer counts each multiset once

Process coins one at a time. After incorporating the first i coins, $dp[s]$ counts the combinations of *those coins only* that sum to s . Adding coin c_i updates $dp[s] += dp[s - c_i]$ for increasing s , which appends copies of c_i to combinations already built from coins $\leq i$. Because a combination is built in a fixed coin order, each multiset is generated exactly once – never as a reordering.

Coin-Outer Loop Counts Combinations

Freezing the coin order and only extending forward eliminates permutations of the same multiset. This is the exact mirror of Coin Combinations I: same array update, loops swapped. Coin-outer $\Rightarrow$ combinations; sum-outer $\Rightarrow$ permutations.

Algorithm

1. $dp[0] = 1$.
2. For each coin c (outer): for $s = c \dots x$ (inner), $dp[s] += dp[s - c]$ modulo $10^9 + 7$.
3. Answer $dp[x]$.

Dry run

Dry Run. Coins $\{2, 3, 5\}$, $x = 9$.

- After coin 2: $dp[\text{even}] = 1$ for $0, 2, 4, 6, 8$.

- After coin 3: combinations of $\{2, 3\}$; e.g. $dp[5] = 1$ ($2+3$), $dp[9]$ grows.
- After coin 5: final $dp[9] = 3$ – the multisets $\{2, 2, 5\}$, $\{2, 2, 2, 3\}$, $\{3, 3, 3\}$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int MOD = 1e9 + 7;
4
5 int main() {
6     int n, x;
7     cin >> n >> x;
8     vector<int> coins(n);
9     for (int& c : coins) cin >> c;
10
11     vector<long long> dp(x + 1, 0);
12     dp[0] = 1;
13     for (int c : coins) // coin outer -> combinations
14         for (int s = c; s <= x; s++)
15             dp[s] = (dp[s] + dp[s - c]) % MOD;
16
17     cout << dp[x] << "\n";
18     return 0;
19 }

```

Complexity Analysis

Time Complexity. $O(nx)$. **Space Complexity.** $O(x)$.

Key Takeaway

Combinations vs permutations in coin DP is decided by *which loop is outer*, nothing else. Keep the pair Coin Combinations I and II side by side in memory as the canonical illustration – the same one-line update, opposite loop nesting, opposite meaning.

1.6 Removing Digits

Problem Statement. Start from an integer n . On each step you may subtract any one of the digits currently appearing in the number. Find the minimum number of steps to reduce n to 0.

Pattern Recognition

“Minimum steps to reduce a number to zero” is a shortest-path DP over values $0 \dots n$. Let $dp[v]$ be the fewest steps from v to 0; from v you may subtract any digit of v , so $dp[v] = 1 + \min_{d \in \text{digits}(v)} dp[v - d]$.

State and transition

$dp[v]$ = minimum steps to bring v down to 0, with $dp[0] = 0$. The only moves from v subtract one of v ’s own decimal digits. Since a larger digit removes more in one step but the available digits depend on v itself, we must try every digit of v and take the minimum successor:

$$dp[v] = 1 + \min_{d \in \text{digits}(v), d > 0} dp[v - d].$$

Greedy “Subtract the Largest Digit” Also Works

Because subtracting a larger digit never overshoots ($v - d \geq 0$ always, as $d \leq 9 \leq v$ for $v \geq 10$, and single digits vanish in one step), a greedy that always removes the largest digit is optimal here. The DP is shown because it generalises and needs no proof of the greedy exchange argument.

Algorithm

1. $dp[0] = 0$.
2. For $v = 1 \dots n$: for each non-zero digit d of v , $dp[v] = \min(dp[v], dp[v - d] + 1)$.
3. Answer $dp[n]$.

Dry run

Dry Run. $n = 27$.

- From 27 subtract 7 $\rightarrow$ 20; from 20 subtract 2 $\rightarrow$ 18; then 8 $\rightarrow$ 10; then 1 $\rightarrow$ 9; then 9 $\rightarrow$ 0.
- The DP finds $dp[27] = 5$ via the value chain $27 \rightarrow 20 \rightarrow 18 \rightarrow 10 \rightarrow 9 \rightarrow 0$ (each step removes one digit).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7     vector<int> dp(n + 1, 1e9);
8     dp[0] = 0;
9     for (int v = 1; v <= n; v++) {
10         int t = v;
11         while (t > 0) {
12             int d = t % 10;
13             if (d > 0) dp[v] = min(dp[v], dp[v - d] + 1);
14             t /= 10;
15         }
16     }
17     cout << dp[n] << "\n";
18     return 0;
19 }
```

Complexity Analysis

Time Complexity. $O(n \log_{10} n)$ – each value inspects its $O(\log n)$ digits. **Space Complexity.** $O(n)$ for the DP array.

Key Takeaway

When moves from a state depend on the state’s own structure (here, its digits), enumerate those moves inside the transition. This “value as a node, allowed subtractions as edges” framing turns a reduce-to-zero puzzle into a one-dimensional shortest-path DP.

1.7 Grid Paths

Problem Statement. Given an $n \times n$ grid where some cells are traps (*) and the rest are free (.), count the number of paths from the top-left to the bottom-right cell moving only right or down, never through a trap. Output modulo $10^9 + 7$.

Pattern Recognition

“Count paths in a grid, moving right/down only” is the textbook 2D grid DP. A cell (i, j) is reached only from directly above or directly left, so $dp[i][j] = dp[i-1][j] + dp[i][j-1]$, with traps forced to 0.

State and transition

$dp[i][j]$ = number of valid paths from $(0, 0)$ to (i, j) . Since the only moves are right and down, the last step into (i, j) came from $(i-1, j)$ (a down move) or $(i, j-1)$ (a right move). Summing those two counts every path once. A trap cell contributes zero paths, so we set its DP value to 0 regardless.

$$dp[i][j] = \begin{cases} 0 & (i, j) \text{ is a trap} \\ dp[i-1][j] + dp[i][j-1] & \text{otherwise.} \end{cases}$$

Two Incoming Directions, One Sum

Grid DP works because the move set is acyclic and monotone: you can only increase i or j , so processing rows top-to-bottom, columns left-to-right, guarantees both predecessors are already computed. Traps are handled by zeroing, which naturally propagates “no path through here.”

Algorithm

1. $dp[0][0] = 1$ if the start is free, else 0.
2. For each cell in row-major order: if it is a trap set 0; else add the values from above and from the left (treating out-of-bounds as 0).
3. Answer $dp[n-1][n-1]$.

Dry run

Dry Run. 3×3 grid with a trap at the centre $(1, 1)$.

- Row 0: 1, 1, 1. Column 0: 1, 1, 1.
- $(1, 1)$ is a trap $\rightarrow 0$. $(1, 2) = dp[0][2] + dp[1][1] = 1 + 0 = 1$. $(2, 1) = 0 + 1 = 1$.
- $(2, 2) = dp[1][2] + dp[2][1] = 1 + 1 = 2$.

C++ implementation

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 const int MOD = 1e9 + 7;
4
5 int main() {
```

```

6   int n;
7   cin >> n;
8   vector<string> g(n);
9   for (auto& row : g) cin >> row;
10
11  vector<vector<long long>> dp(n, vector<long long>(n, 0));
12  for (int i = 0; i < n; i++)
13      for (int j = 0; j < n; j++) {
14          if (g[i][j] == '*') { dp[i][j] = 0; continue; }
15          if (i == 0 && j == 0) { dp[i][j] = 1; continue; }
16          long long up = (i > 0) ? dp[i - 1][j] : 0;
17          long long left = (j > 0) ? dp[i][j - 1] : 0;
18          dp[i][j] = (up + left) % MOD;
19      }
20  cout << dp[n - 1][n - 1] << "\n";
21  return 0;
22 }

```

Complexity Analysis

Time Complexity. $O(n^2)$ – one pass over the grid. **Space Complexity.** $O(n^2)$, reducible to $O(n)$ by keeping only the previous row.

Key Takeaway

Grid path counting is the 2D analogue of the 1D coin sums: sum over the directions the last move could have come from. Obstacles become zeros that propagate automatically. This (i, j) -state template underlies Edit Distance and Counting Tilings later in the chapter.

1.8 Book Shop

Problem Statement. You have x money and there are n books, each with a price and a page count. You may buy each book at most once. Maximise the total number of pages you can buy within your budget.

Pattern Recognition

“Each item at most once, maximise value under a capacity” is the classic **0/1 knapsack**. Price is weight, pages are value, x is capacity. Let $dp[b]$ be the maximum pages achievable with budget exactly $\leq b$.

State, transition, and the reverse inner loop

Process books one at a time. $dp[b]$ = best pages using books considered so far with budget b . For a book with price p and pages s , either skip it ($dp[b]$ unchanged) or buy it ($dp[b - p] + s$). The decisive implementation detail: when using a 1D array, iterate b **downward** from x to p , so that $dp[b - p]$ still refers to a state *without* the current book – guaranteeing each book is used at most once.

$$dp[b] = \max(dp[b], dp[b - p] + s).$$

Downward Loop Enforces “At Most Once”

In 0/1 knapsack the inner budget loop must go high-to-low. A low-to-high loop would let $dp[b - p]$ already include the current book, effectively allowing it to be bought repeatedly –

which is the unbounded knapsack (Coin Combinations II). The loop direction is the whole difference.

Algorithm

1. $dp[b] = 0$ for all b .
2. For each book (p, s) : for $b = x$ down to p , $dp[b] = \max(dp[b], dp[b - p] + s)$.
3. Answer $dp[x]$.

Dry run

Dry Run. Budget $x = 10$; books (price, pages) = (4, 5), (8, 12), (5, 8), (3, 1).

- After book (4, 5): $dp[b] = 5$ for $b \geq 4$.
- After (5, 8): $dp[9] = \max(5, dp[4] + 8) = 13$ (buy the 4 and the 5).
- Final $dp[10] = 13$ – books priced 4 and 5 give $5 + 8 = 13$ pages within budget 10.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, x;
6     cin >> n >> x;
7     vector<int> price(n), pages(n);
8     for (int& p : price) cin >> p;
9     for (int& s : pages) cin >> s;
10
11     vector<int> dp(x + 1, 0);
12     for (int i = 0; i < n; i++)
13         for (int b = x; b >= price[i]; b--)           // downward: 0/1
14             dp[b] = max(dp[b], dp[b - price[i]] + pages[i]);
15
16     cout << dp[x] << "\n";
17     return 0;
18 }
```

Complexity Analysis

Time Complexity. $O(nx)$ – each book sweeps the budget once. **Space Complexity.** $O(x)$ with the rolling 1D array.

Key Takeaway

Book Shop is 0/1 knapsack in disguise; the only thing to get right is the downward inner loop that forbids reusing an item. Contrast with the coin problems' upward loop (items reusable). Loop direction is the recurring knapsack tell.

1.9 Array Description

Problem Statement. An array of n integers, each between 1 and m , is partially known: some positions hold a value, others are 0 (unknown). Count the ways to fill the unknowns so that adjacent elements differ by at most 1. Output modulo $10^9 + 7$.

Pattern Recognition

“Count fillings where adjacent elements are constrained” is a DP where the state carries the value at the current position. Let $dp[i][v]$ be the number of ways to fill the prefix ending with value v at position i ; the next position may hold $v - 1, v$, or $v + 1$.

State and transition

$dp[i][v]$ = ways to fill positions $1 \dots i$ such that position i equals v and all adjacency constraints hold. From a valid prefix ending in value u at $i - 1$, position i may take $u - 1, u, u + 1$; equivalently a value v at i can follow $v - 1, v, v + 1$ at $i - 1$:

$$dp[i][v] = dp[i - 1][v - 1] + dp[i - 1][v] + dp[i - 1][v + 1].$$

If position i is fixed to some $a_i \neq 0$, only $v = a_i$ is allowed (all other $dp[i][v] = 0$); if it is 0, every $v \in [1, m]$ is allowed.

Carry the Last Value in the State

Adjacency constraints couple neighbours, so the DP state must remember the value just placed. With that, each transition is a local three-way sum – the ± 1 neighbourhood – and fixed positions simply prune the allowed v .

Algorithm

1. Initialise row 1: if a_1 fixed, $dp[1][a_1] = 1$; else $dp[1][v] = 1$ for all v .
2. For $i = 2 \dots n$ and each allowed v at i : $dp[i][v] = dp[i - 1][v - 1] + dp[i - 1][v] + dp[i - 1][v + 1]$.
3. Answer $\sum_{v=1}^m dp[n][v]$.

Dry run

Dry Run. $n = 3, m = 5$, array $[2, 0, 2]$.

- Row 1: only $dp[1][2] = 1$.
- Row 2 (0, free): $dp[2][1] = dp[1][2] = 1$ (from 2), $dp[2][2] = 1$, $dp[2][3] = 1$; others 0.
- Row 3 fixed to 2: $dp[3][2] = dp[2][1] + dp[2][2] + dp[2][3] = 3$. Answer 3.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int MOD = 1e9 + 7;
4
5 int main() {
6     int n, m;
7     cin >> n >> m;
```

```

8   vector<int> a(n + 1);
9   for (int i = 1; i <= n; i++) cin >> a[i];
10
11  vector<vector<long long>> dp(n + 1, vector<long long>(m + 2, 0));
12  for (int v = 1; v <= m; v++)
13      if (a[1] == 0 || a[1] == v) dp[1][v] = 1;
14
15  for (int i = 2; i <= n; i++)
16      for (int v = 1; v <= m; v++) {
17          if (a[i] != 0 && a[i] != v) continue;           // fixed
18              position
19              dp[i][v] = (dp[i-1][v-1] + dp[i-1][v] + dp[i-1][v+1]) % MOD;
20          }
21
22  long long ans = 0;
23  for (int v = 1; v <= m; v++) ans = (ans + dp[n][v]) % MOD;
24  cout << ans << "\n";
25  return 0;
}

```

Complexity Analysis

Time Complexity. $O(nm)$ – n positions times m possible values, each an $O(1)$ transition.

Space Complexity. $O(nm)$, reducible to $O(m)$ with two rolling rows.

Key Takeaway

When adjacent elements constrain each other, promote the “current value” into the DP state; the transition then only touches the local neighbourhood the constraint allows. Fixed inputs prune states rather than complicate the recurrence.

1.10 Counting Towers

Problem Statement. Build a tower of width 2 and height n out of blocks that are 1×1 or 1×2 (blocks may be placed in either orientation to fill the 2-wide grid). Count the number of distinct towers of height n , modulo $10^9 + 7$. Answer T test cases.

Pattern Recognition

This is a row-by-row DP with a two-valued state describing the **boundary shape** between consecutive levels. At each height the seam between the level below and above is either *split* (two separate cells) or *joined* (one 1×2 block spanning the width). Transitions between these two boundary states, counted with fixed multipliers, give the recurrence.

Two boundary states

Process the tower one unit of height at a time, tracking the shape of the horizontal seam at the current top:

- State 0 – the seam is **split** into two independent cells (the two columns are separable at this height).
- State 1 – the seam is **joined** by a single horizontal 1×2 block spanning both columns.

Counting how each state can be extended upward by one level yields:

$$\begin{aligned} dp[i][0] &= 4 dp[i-1][0] + 2 dp[i-1][1], \\ dp[i][1] &= dp[i-1][0] + dp[i-1][1]. \end{aligned}$$

The constants 4 and 2 enumerate the ways to fill one more level given the seam below (four fillings keep the seam split, etc.). Base: $dp[1][0] = dp[1][1] = 1$.

The State Is the Seam, Not the Whole Tower

The only information the next level needs from everything below is the *shape of the current top seam*. Compressing the entire history into one of two boundary states is what makes this an $O(n)$ DP rather than an exponential enumeration – a miniature “broken profile” idea, revisited fully in Counting Tilings (Section 1.19).

Algorithm

1. Precompute $dp[i][0]$, $dp[i][1]$ for $i = 1 \dots \max n$ using the recurrence above.
2. For each test case n , answer $dp[n][0] + dp[n][1]$ modulo $10^9 + 7$.

Dry run

Dry Run. Small heights.

- $n = 1$: $dp[1][0] = 1$, $dp[1][1] = 1$; total 2 (two cells, or one horizontal block).
- $n = 2$: $dp[2][0] = 4 \cdot 1 + 2 \cdot 1 = 6$, $dp[2][1] = 1 + 1 = 2$; total 8.
- $n = 3$: $dp[3][0] = 4 \cdot 6 + 2 \cdot 2 = 28$, $dp[3][1] = 6 + 2 = 8$; total 36.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int MOD = 1e9 + 7;
4 const int MAXN = 1e6 + 1;
5
6 int main() {
7     static long long dp0[MAXN], dp1[MAXN];
8     dp0[1] = dp1[1] = 1;
9     for (int i = 2; i < MAXN; i++) {
10         dp0[i] = (4 * dp0[i-1] + 2 * dp1[i-1]) % MOD;
11         dp1[i] = (dp0[i-1] + dp1[i-1]) % MOD;
12     }
13
14     int t;
15     cin >> t;
16     while (t--) {
17         int n;
18         cin >> n;
19         cout << (dp0[n] + dp1[n]) % MOD << "\n";
20     }
21     return 0;
22 }
```

Complexity Analysis

Time Complexity. $O(\max n)$ precomputation, then $O(1)$ per query. **Space Complexity.** $O(\max n)$ for the two DP arrays.

Key Takeaway

When a structure is built layer by layer, the DP state need only capture the *interface* between the built and unbuilt parts. Here two seam states suffice; identifying that minimal boundary description is the core skill, scaling up to bitmask profiles for wider grids.

1.11 Edit Distance

Problem Statement. Given two strings, find their edit (Levenshtein) distance: the minimum number of single-character insertions, deletions, and substitutions needed to turn one string into the other.

Pattern Recognition

“Minimum operations to transform one sequence into another” is the canonical two-sequence grid DP. Let $dp[i][j]$ be the edit distance between the first i characters of A and the first j of B ; the last operation is an insert, delete, or match/substitute.

State and transition

$dp[i][j]$ = edit distance between prefixes $A[1..i]$ and $B[1..j]$. Consider the last operation aligning the two prefixes:

- If $A[i] = B[j]$, they match for free: $dp[i-1][j-1]$.
- Otherwise substitute: $1 + dp[i-1][j-1]$.
- Delete $A[i]$: $1 + dp[i-1][j]$.
- Insert $B[j]$: $1 + dp[i][j-1]$.

$$dp[i][j] = \min(dp[i-1][j-1] + [A_i \neq B_j], dp[i-1][j] + 1, dp[i][j-1] + 1).$$

Base cases: $dp[i][0] = i$ (delete all), $dp[0][j] = j$ (insert all).

Three Operations, Three Predecessors

Each cell chooses the cheapest of the three edits that could have produced it, so the recurrence has exactly three incoming states. The diagonal move is free on a character match – that is where the strings’ similarity saves operations.

Algorithm

1. Fill the first row and column with $0, 1, 2, \dots$ (transform to or from the empty string).
2. For $i = 1 \dots |A|$, $j = 1 \dots |B|$: apply the recurrence above.
3. Answer $dp[|A|][|B|]$.

Dry run

Dry Run. $A = \text{"LOVE"}, B = \text{"MOVIE"}$.

- Align: $L \rightarrow M$ (substitute), $O=O$, $V=V$, insert I , $E=E$.
- The DP settles $dp[4][5] = 2$ – one substitution ($L \rightarrow M$) and one insertion (I).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     string a, b;
6     cin >> a >> b;
7     int n = a.size(), m = b.size();
8
9     vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
10    for (int i = 0; i <= n; i++) dp[i][0] = i;
11    for (int j = 0; j <= m; j++) dp[0][j] = j;
12
13    for (int i = 1; i <= n; i++)
14        for (int j = 1; j <= m; j++) {
15            int cost = (a[i - 1] == b[j - 1]) ? 0 : 1;
16            dp[i][j] = min({ dp[i-1][j-1] + cost,      // match/substitute
17                           dp[i-1][j] + 1,           // delete
18                           dp[i][j-1] + 1 });        // insert
19        }
20    cout << dp[n][m] << "\n";
21    return 0;
22 }
```

Complexity Analysis

Time Complexity. $O(nm)$. **Space Complexity.** $O(nm)$, reducible to $O(\min(n, m))$ with two rolling rows.

Key Takeaway

Edit Distance is the two-string grid DP every alignment problem descends from: three edit operations map to three predecessor cells, and a matching character makes the diagonal free. The same (i, j) -prefix state solves LCS, sequence alignment, and diff.

1.12 Rectangle Cutting

Problem Statement. Given an $a \times b$ rectangle, repeatedly cut it with straight horizontal or vertical cuts (each cut splits a piece into two, along integer coordinates) until only squares remain. Find the minimum number of cuts needed.

Pattern Recognition

“Minimum cuts to reduce a rectangle to squares” is an interval DP in two dimensions. Let $dp[h][w]$ be the fewest cuts for an $h \times w$ rectangle; a first cut splits it into two smaller rectangles whose costs are already known.

State and transition

$dp[h][w]$ = minimum cuts for an $h \times w$ piece. If $h = w$ it is already a square: $dp[h][h] = 0$. Otherwise the first cut is either horizontal (splitting height into i and $h - i$) or vertical (splitting width into j and $w - j$), each costing one plus the two sub-costs:

$$dp[h][w] = 1 + \min\left(\min_{1 \leq i < h} (dp[i][w] + dp[h - i][w]), \min_{1 \leq j < w} (dp[h][j] + dp[h][w - j])\right).$$

Try Every First Cut

As in all interval DP, we do not guess the optimal cut – we try every possible first cut and take the minimum, trusting that sub-rectangles are solved optimally. Squares terminate the recursion at zero cost.

Algorithm

1. For all s , $dp[s][s] = 0$.
2. For increasing h, w with $h \neq w$: take the best over all horizontal splits i and vertical splits j , add 1.
3. Answer $dp[a][b]$.

Dry run

Dry Run. $a = 3, b = 5$.

- $dp[3][3] = 0$. $dp[2][3] = 1 + \min \text{ over cuts} = 1 + dp[2][2] + dp[2][1]$ -type options = 2 (cut off a 2×2 , then split the 2×1).
- Working up, $dp[3][5] = 3$ – e.g. cut a 3×5 into 3×3 and 3×2 , then the 3×2 into a 2×2 and pieces, totalling three cuts.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int a, b;
6     cin >> a >> b;
7     vector<vector<int>> dp(a + 1, vector<int>(b + 1, 0));
8
9     for (int h = 1; h <= a; h++)
10         for (int w = 1; w <= b; w++) {
11             if (h == w) { dp[h][w] = 0; continue; }
12             int best = INT_MAX;
13             for (int i = 1; i < h; i++) // horizontal
14                 cut
15                 best = min(best, dp[i][w] + dp[h - i][w]);
16             for (int j = 1; j < w; j++) // vertical cut
17                 best = min(best, dp[h][j] + dp[h][w - j]);
18             dp[h][w] = best + 1;
19         }
20     cout << dp[a][b] << "\n";
21     return 0;
22 }
```

Complexity Analysis

Time Complexity. $O(ab(a+b))$ – each of the ab states tries $O(a+b)$ cut positions. **Space Complexity.** $O(ab)$.

Key Takeaway

Interval DP in 2D: enumerate every first cut, recurse on the two pieces, take the minimum. The “try all split points” pattern recurs in Removal Game and any problem where a structure is optimally divided into independent parts.

1.13 Money Sums

Problem Statement. You have n coins with given values. Determine all the distinct sums you can form using *some subset* of the coins (each coin used at most once). Output how many distinct sums are possible and list them.

Pattern Recognition

“Which totals are achievable from a subset” is the boolean **subset-sum** DP. Let $dp[s]$ be true if some subset sums to exactly s ; each coin either joins a subset or not, a 0/1 choice.

State and transition

$dp[s]$ = whether sum s is reachable using a subset of the coins seen so far, with $dp[0] = \text{true}$ (the empty subset). Adding a coin of value c makes any previously reachable s also reach $s + c$. Using a 1D array we iterate s **downward** (0/1 knapsack rule) so a coin is not reused:

$$dp[s] \leftarrow dp[s] \vee dp[s - c].$$

Reachability, Not Counting

Money Sums only asks *whether* a sum is attainable, so the DP stores booleans and combines with OR. The total possible sum is $\sum c_i \leq 10^5$, so a single boolean array over $[0, \text{total}]$ suffices, and the answer is every index set to true.

Algorithm

1. Let $S = \sum c_i$. Set $dp[0] = \text{true}$.
2. For each coin c : for $s = S$ down to c , $dp[s] \leftarrow dp[s] \vee dp[s - c]$.
3. Collect all $s \geq 1$ with $dp[s]$ true; their count and list are the answer.

Dry run

Dry Run. Coins $\{4, 2, 2\}$; total $S = 8$.

- Start $dp[0] = \text{T}$. After coin 4: $\{0, 4\}$.
- After a 2: $\{0, 2, 4, 6\}$. After the second 2: $\{0, 2, 4, 6, 8\}$.
- Non-zero reachable sums: 2, 4, 6, 8 – four distinct sums.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7     vector<int> coins(n);
8     int total = 0;
9     for (int& c : coins) { cin >> c; total += c; }
10
11     vector<char> dp(total + 1, 0);
12     dp[0] = 1;
13     for (int c : coins)
14         for (int s = total; s >= c; s--)           // downward: 0/1
15             if (dp[s - c]) dp[s] = 1;
16
17     vector<int> sums;
18     for (int s = 1; s <= total; s++) if (dp[s]) sums.push_back(s);
19
20     cout << sums.size() << "\n";
21     for (int s : sums) cout << s << " ";
22     cout << "\n";
23     return 0;
24 }

```

Complexity Analysis

Time Complexity. $O(n \cdot S)$ where $S = \sum c_i$. **Space Complexity.** $O(S)$ for the boolean array.

Key Takeaway

Subset-sum reachability is 0/1 knapsack with booleans and OR. The downward loop again enforces “each coin once.” This reachability array is the engine behind Two Sets II (next), where we *count* the subsets rather than merely test existence.

1.14 Removal Game

Problem Statement. An array of n numbers is on a table. Two players alternate; on each turn a player removes either the leftmost or rightmost element and adds its value to their score. Both play optimally to maximise their own score. Find the first player’s final score.

Pattern Recognition

“Two players take from the ends, both optimal” is an interval game DP. Let $dp[l][r]$ be the best score *difference* (current player minus opponent) achievable on the subarray $[l, r]$; the current player takes an end and then *becomes* the opponent on the rest.

State and transition via score difference

Track $dp[l][r]$ = the maximum of (current player’s score – opponent’s score) over the subarray $a[l..r]$, assuming optimal play. The current player takes $a[l]$ or $a[r]$; after that the opponent

plays optimally on the remaining interval, so their advantage is subtracted:

$$dp[l][r] = \max(a[l] - dp[l+1][r], a[r] - dp[l][r-1]), \quad dp[l][l] = a[l].$$

If $total = \sum a_i$ and $d = dp[0][n-1]$ is the final difference, the first player's score is $(total + d)/2$.

Score Difference Turns Two Objectives into One

Modelling the game by the *difference* folds “maximise mine” and “opponent minimises mine” into a single recurrence: whatever the opponent gains counts against me, so I subtract their optimal advantage on the remainder. This zero-sum trick avoids storing two separate scores.

Algorithm

1. $dp[i][i] = a[i]$.
2. For increasing interval length, $dp[l][r] = \max(a[l] - dp[l+1][r], a[r] - dp[l][r-1])$.
3. First player's score = $(total + dp[0][n-1])/2$.

Dry run

Dry Run. $a = [4, 5, 1, 3]$, total = 13.

- Length 1: $dp[i][i] = a[i]$.
- Length 2: $dp[0][1] = \max(4 - 5, 5 - 4) = 1$; $dp[2][3] = \max(1 - 3, 3 - 1) = 2$; $dp[1][2] = \max(5 - 1, 1 - 5) = 4$.
- Full: $dp[0][3] = \max(4 - dp[1][3], 3 - dp[0][2])$. With $dp[1][3] = \max(5 - 2, 3 - 4) = 3$ and $dp[0][2] = \max(4 - 4, 1 - 1) = 0$: $\max(4 - 3, 3 - 0) = 3$.
- First player's score = $(13 + 3)/2 = 8$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7     vector<long long> a(n);
8     long long total = 0;
9     for (auto& x : a) { cin >> x; total += x; }
10
11     vector<vector<long long>> dp(n, vector<long long>(n, 0));
12     for (int i = 0; i < n; i++) dp[i][i] = a[i];
13
14     for (int len = 2; len <= n; len++)
15         for (int l = 0; l + len - 1 < n; l++) {
16             int r = l + len - 1;
17             dp[l][r] = max(a[l] - dp[l+1][r], a[r] - dp[l][r-1]);
18         }
19
20     cout << (total + dp[0][n-1]) / 2 << "\n";
21     return 0;
22 }
```

Complexity Analysis

Time Complexity. $O(n^2)$ – one $O(1)$ transition per interval. **Space Complexity.** $O(n^2)$ for the interval table.

Key Takeaway

For symmetric two-player, zero-sum games, track the *score difference* from the mover's perspective and subtract the opponent's optimal remainder. This collapses adversarial optimisation into a single maximisation – the standard move for end-taking and Nim-like game DPs.

1.15 Two Sets II

Problem Statement. Count the number of ways to partition the numbers $\{1, 2, \dots, n\}$ into two subsets with *equal* sums. Two partitions that are reflections of each other count as the same. Output modulo $10^9 + 7$.

Pattern Recognition

“Split $\{1..n\}$ into two equal-sum halves, count the ways” is a **counting subset-sum**. The total is $T = \frac{n(n+1)}{2}$; if T is odd the answer is 0, else we count subsets summing to $T/2$, then divide by 2 to remove the mirror duplication.

State and transition

Let the target be $t = T/2$. Define $dp[s]$ = number of subsets of $\{1, \dots, n\}$ (processed one number at a time) that sum to s , with $dp[0] = 1$. Each number k is either in the subset or not – a 0/1 choice – so with the downward inner loop:

$$dp[s] += dp[s - k] \quad \text{for } s = t \text{ down to } k.$$

Each valid partition is counted twice (subset A and its complement B), so the final answer is $dp[t]/2$ modulo $10^9 + 7$ (using the modular inverse of 2).

Count Subsets, Then Halve for the Mirror

Counting subsets that reach exactly half the total counts every partition from both sides. Dividing by 2 (via the modular inverse, since we are mod a prime) removes that symmetry. The odd-total shortcut avoids pointless work when no equal split exists.

Algorithm

1. Compute $T = n(n+1)/2$; if T is odd, output 0.
2. Set $t = T/2$, $dp[0] = 1$. For $k = 1 \dots n - 1$ (one number may be fixed to a side to pre-halve, or process all and divide): update $dp[s] += dp[s - k]$ downward.
3. Output $dp[t]$, halved via the modular inverse of 2.

(Alternatively, process $k = 1 \dots n - 1$ only – fixing n to one side – which directly avoids the double count without a division.)

Dry run

Dry Run. $n = 7$, $T = 28$, target $t = 14$.

- Subsets of $\{1..7\}$ summing to 14 include $\{7, 6, 1\}$, $\{7, 5, 2\}$, $\{7, 4, 3\}$, $\{7, 4, 2, 1\}$, ...
- The counting DP gives $dp[14] = 8$; halving yields the answer 4 distinct partitions.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const long long MOD = 1e9 + 7;
4
5 int main() {
6     int n;
7     cin >> n;
8     long long total = (long long)n * (n + 1) / 2;
9     if (total % 2 != 0) { cout << 0 << "\n"; return 0; }
10
11     int t = total / 2;
12     vector<long long> dp(t + 1, 0);
13     dp[0] = 1;
14     // fix element n on one side: process 1..n-1, no division needed
15     for (int k = 1; k <= n - 1; k++)
16         for (int s = t; s >= k; s--)
17             dp[s] = (dp[s] + dp[s - k]) % MOD;
18
19     cout << dp[t] << "\n";
20     return 0;
21 }

```

Complexity Analysis

Time Complexity. $O(n \cdot T) = O(n^3)$ in the worst sense, but $T = O(n^2)$ so it is $O(n \cdot n^2)$; practically $O(n \cdot t)$ with $t \approx n^2/4$. **Space Complexity.** $O(t) = O(n^2)$.

Key Takeaway

Two Sets II is counting subset-sum with a symmetry fix. The clean way to avoid the double count is to fix one fixed element (here n) to a single side and only distribute the rest – turning “divide by two” into “count directly.” Recognise equal-partition problems as subset-sum to half the total.

1.16 Increasing Subsequence

Problem Statement. Given an array of n integers, find the length of the longest strictly increasing subsequence (LIS): the longest sequence of indices $i_1 < i_2 < \dots$ with strictly increasing values.

Pattern Recognition

“Longest increasing subsequence” has an elegant $O(n \log n)$ solution built on **patience sorting**: maintain the smallest possible tail value for an increasing subsequence of each length, and binary-search where each new element fits.

The tails array

Keep an array *tails*, where *tails*[*k*] is the smallest value that can end an increasing subsequence of length *k* + 1 seen so far. This array is always sorted. For each new value *x*, find the first tail $\geq x$ (a `lower_bound`) and overwrite it with *x*: this either extends the longest subsequence (if *x* is larger than every tail, append) or improves a length-*k* subsequence's ending to a smaller value, leaving room for future growth. The LIS length is the final size of *tails*.

Keep the Smallest Tail per Length

Replacing the first tail $\geq x$ with *x* never shortens any achievable length and keeps every tail as small as possible, maximising the chance of future extension. The binary search is what turns the naive $O(n^2)$ DP into $O(n \log n)$.

Algorithm

1. Start with an empty *tails*.
2. For each *x*: binary-search the first index with *tails*[*i*] $\geq x$. If none, append *x* (LIS grew); else set *tails*[*i*] = *x*.
3. Answer is the size of *tails*.

Dry run

Dry Run. *a* = [7, 3, 5, 3, 6, 2, 9, 8].

- 7 $\rightarrow$ [7]; 3 replaces 7 $\rightarrow$ [3]; 5 appends $\rightarrow$ [3, 5]; 3 replaces 5? No – `lower_bound` of 3 is index 0 ($3 \geq 3$), replace $\rightarrow$ [3, 5] (unchanged in effect).
- 6 appends $\rightarrow$ [3, 5, 6]; 2 replaces 3 $\rightarrow$ [2, 5, 6]; 9 appends $\rightarrow$ [2, 5, 6, 9]; 8 replaces 9 $\rightarrow$ [2, 5, 6, 8].
- LIS length = 4 (e.g. 3, 5, 6, 9).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7     vector<int> a(n);
8     for (int& x : a) cin >> x;
9
10    vector<int> tails;
11    for (int x : a) {
12        auto it = lower_bound(tails.begin(), tails.end(), x); // first
13                       >= x
14        if (it == tails.end()) tails.push_back(x); // extend LIS
15        else *it = x; // shrink a tail
16    }
17    cout << tails.size() << "\n";
18    return 0;
19 }
```

Complexity Analysis

Time Complexity. $O(n \log n)$ – one binary search per element. **Space Complexity.** $O(n)$ for the tails array. (A simpler $O(n^2)$ DP with $dp[i] = 1 + \max_{j < i, a_j < a_i} dp[j]$ also solves it for small n .)

Key Takeaway

LIS in $O(n \log n)$ is the patience-sorting tails trick: keep the minimal ending value for each length, binary-search each insertion. For strictly increasing use `lower_bound`; for non-decreasing use `upper_bound` – a one-word change that trips up many.

1.17 Projects

Problem Statement. There are n projects, each with a start day, an end day, and a reward. Attending a project occupies all days from its start to its end inclusive, so two attended projects may not overlap. Maximise the total reward.

Pattern Recognition

“Non-overlapping intervals, maximise total value” is **weighted interval scheduling**. Sort projects by end day; for each, either skip it or take it and jump to the best solution that finishes before its start – found by binary search.

State and transition

Sort the projects by end day. Let $dp[i]$ be the maximum reward using only the first i projects (in sorted order). For project i with start s_i and reward p_i , either skip it ($dp[i - 1]$) or take it: add p_i to the best reward achievable among projects that end *strictly before* s_i . Let j be the number of projects whose end day $< s_i$ (found by binary search on the sorted end days):

$$dp[i] = \max(dp[i - 1], p_i + dp[j]).$$

Sort by End, Binary-Search the Predecessor

Sorting by end day means that when we consider a project, every compatible earlier project has already been folded into some $dp[j]$. Binary search finds the latest non-conflicting project in $O(\log n)$, turning an $O(n^2)$ scan into $O(n \log n)$.

Algorithm

1. Sort projects by end day; let `ends[]` be their end days.
2. $dp[0] = 0$. For $i = 1 \dots n$: binary-search $j = \text{number of projects with end} < s_i$; set $dp[i] = \max(dp[i - 1], p_i + dp[j])$.
3. Answer $dp[n]$.

Dry run

Dry Run. Projects (start, end, reward): (2, 4, 7), (1, 2, 4), (5, 6, 3).

- Sorted by end: (1, 2, 4), (2, 4, 7), (5, 6, 3).

- $dp[1] = 4$. Project (2, 4, 7): projects ending < 2 is the first (end 2? no, need < 2 , so $j = 0$): $dp[2] = \max(4, 7 + 0) = 7$.
- Project (5, 6, 3): projects ending < 5 are both, $j = 2$: $dp[3] = \max(7, 3 + 7) = 10$. Answer 10.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7     vector<array<long long, 3>> p(n);           // {end, start, reward}
8     for (int i = 0; i < n; i++) {
9         long long a, b, x;
10        cin >> a >> b >> x;
11        p[i] = {b, a, x};
12    }
13    sort(p.begin(), p.end());                  // by end day
14
15    vector<long long> ends(n);
16    for (int i = 0; i < n; i++) ends[i] = p[i][0];
17
18    vector<long long> dp(n + 1, 0);
19    for (int i = 1; i <= n; i++) {
20        long long start = p[i-1][1], reward = p[i-1][2];
21        // number of projects with end < start
22        int j = lower_bound(ends.begin(), ends.end(), start) - ends.
                begin();
23        dp[i] = max(dp[i-1], reward + dp[j]);
24    }
25    cout << dp[n] << "\n";
26    return 0;
27 }

```

Complexity Analysis

Time Complexity. $O(n \log n)$ – sorting plus a binary search per project. **Space Complexity.** $O(n)$.

Key Takeaway

Weighted interval scheduling = sort by end, then “skip or take with a binary-searched predecessor.” It generalises the unweighted greedy (which just maximises count) to rewards, where greed fails and DP is required.

1.18 Elevator Rides

Problem Statement. There are n people with given weights, and an elevator with maximum weight x . Find the minimum number of elevator rides needed to carry everyone (people may be grouped arbitrarily, but each ride’s total weight must not exceed x).

Pattern Recognition

With $n \leq 20$, “partition people into fewest groups under a capacity” screams **bitmask DP**: the state is the *subset* of people already transported. The clever part is what to store – not just the ride count, but how full the last ride is.

A two-part state per subset

For each subset *mask* of people, store a pair $dp[mask] = (rides, lastWeight)$: the minimum number of rides to carry exactly that subset, and among solutions with that few rides, the smallest weight currently loaded in the last (still-open) ride. Compare pairs lexicographically – fewer rides first, then a lighter last ride (which leaves the most room). Transition by adding one new person *p* to *mask*:

- If *p* fits in the current last ride ($lastWeight + w_p \leq x$): same ride count, last weight grows by w_p .
- Otherwise start a new ride: rides +1, last weight = w_p .

Store Rides *and* the Last Ride’s Load

Minimising rides alone is not enough, because two subsets with equal ride counts differ in how much room remains – and that room affects future additions. Pairing the ride count with the least-loaded last ride makes the greedy-within-DP choice optimal.

Algorithm

1. $dp[0] = (1, 0)$ – one empty ride, zero load.
2. For each mask and each person $p \notin mask$: compute the candidate pair for $mask \cup \{p\}$ using the rule above; keep the lexicographically smaller.
3. Answer is the ride count of $dp[\text{full mask}]$.

Dry run

Dry Run. Weights $[2, 3, 3]$, capacity $x = 5$.

- $dp[\{0\}] = (1, 2)$, $dp[\{1\}] = (1, 3)$.
- Add person 2 ($w = 3$) to $\{0\}$ (last 2): $2 + 3 = 5 \leq 5$, same ride: $(1, 5)$. To $\{1\}$ (last 3): $3 + 3 = 6 > 5$, new ride: $(2, 3)$.
- $dp[\{0, 2\}] = (1, 5)$, then adding person 1 (3): $5 + 3 > 5$, new ride $(2, 3)$. Full-set answer: 2 rides (e.g. $\{2, 3\}$ and $\{3\}$).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, x;
6     cin >> n >> x;
7     vector<int> w(n);

```

```

8   for (int& v : w) cin >> v;
9
10  int full = 1 << n;
11  // dp[mask] = {rides, weight in last ride}
12  vector<pair<int,int>> dp(full, {n + 1, 0});
13  dp[0] = {1, 0};
14
15  for (int mask = 1; mask < full; mask++) {
16      for (int p = 0; p < n; p++) {
17          if (!(mask & (1 << p))) continue;
18          auto prev = dp[mask ^ (1 << p)]; // state without p
19          if (prev.second + w[p] <= x) // fits in last ride
20              prev.second += w[p];
21          else // needs a new ride
22              prev = {prev.first + 1, w[p]};
23          dp[mask] = min(dp[mask], prev); // lexicographic
24      }
25  }
26  cout << dp[full - 1].first << "\n";
27  return 0;
28 }

```

Complexity Analysis

Time Complexity. $O(2^n \cdot n)$ – each of 2^n subsets tries adding each of n people. **Space Complexity.** $O(2^n)$ for the DP table.

Key Takeaway

Bitmask DP fits partition problems with $n \leq 20$. The lesson beyond the mask is the *compound state*: pairing “rides so far” with “room left in the current ride” captures exactly the extra information a greedy packing needs – a recurring trick when a count alone loses too much.

1.19 Counting Tilings

Problem Statement. Count the number of ways to tile an $n \times m$ grid completely with 1×2 dominoes (each placed horizontally or vertically), modulo $10^9 + 7$.

Pattern Recognition

“Domino tilings of a grid” with one small dimension ($m \leq 10$) is solved by **broken-profile (bitmask) DP**: sweep column by column, and let the state be the bitmask of cells in the current column that are already occupied by a horizontal domino protruding from the previous column.

The profile between columns

Fill the grid one column at a time. When moving from column c to column $c + 1$, the only thing the next column needs to know is which of its cells are *already filled* by horizontal dominoes sticking out of column c . That set of n bits is the **profile**. For each incoming profile $mask$, we enumerate every legal way to complete column c using vertical dominoes (within the column) and horizontal dominoes (extending into column $c + 1$), producing an outgoing profile.

Recursively Fill One Column

Process the current column cell by cell from top to bottom. A cell already covered by the incoming profile is skipped. An empty cell must be covered either by a vertical domino with the cell below it, or by a horizontal domino that sets the corresponding bit in the *next* column's profile. Enumerating these choices generates all valid transitions.

Algorithm

1. Ensure n (the profile height) is the smaller dimension; if not, swap n and m .
2. Start with $cur[0] = 1$ (empty profile before column 0).
3. For each column, for each profile with nonzero count, recursively fill the column to produce all next-column profiles, accumulating counts.
4. After the last column, the answer is $cur[0]$ (nothing protrudes past the grid).

Dry run

Dry Run. 2×2 grid.

- Column 0, incoming profile 00: either two horizontals (both cells extend right, next profile 11) or one vertical (next profile 00).
- Column 1: profile 11 means both cells already filled – valid, next 00. Profile 00 needs a vertical – valid, next 00.
- Total tilings of 2×2 : 2 (two horizontals or two verticals).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const long long MOD = 1e9 + 7;
4
5 int n, m;
6 vector<long long> cur, nxt;
7
8 // fill column: row = current row, mask = incoming profile,
9 // newmask = outgoing profile being built, ways = count to propagate
10 void fill(int row, int mask, int newmask, long long ways) {
11     if (row == n) { nxt[newmask] = (nxt[newmask] + ways) % MOD; return;
12     }
13     if (mask & (1 << row)) { // already filled
14         return;
15     }
16     // place a horizontal domino: extends into next column
17     fill(row + 1, mask, newmask | (1 << row), ways);
18     // place a vertical domino covering row and row+1
19     if (row + 1 < n && !(mask & (1 << (row + 1))))
20         fill(row + 2, mask, newmask, ways);
21 }
22
23 int main() {

```

```

24     cin >> n >> m;
25     if (n > m) swap(n, m);           // keep profile small
26     int full = 1 << n;
27     cur.assign(full, 0);
28     cur[0] = 1;
29     for (int col = 0; col < m; col++) {
30         nxt.assign(full, 0);
31         for (int mask = 0; mask < full; mask++)
32             if (cur[mask]) fill(0, mask, 0, cur[mask]);
33         cur = nxt;
34     }
35     cout << cur[0] << "\n";         // nothing protrudes
36     out
37     return 0;
38 }

```

Complexity Analysis

Time Complexity. $O(m \cdot 2^n \cdot n)$ roughly – each column processes 2^n profiles, each filled in $O(n)$ recursive steps. **Space Complexity.** $O(2^n)$ for the two profile arrays.

Key Takeaway

Broken-profile DP tiles a grid by carrying, as state, only the boundary between the filled and unfilled regions – one column’s protrusion mask. It is Counting Towers (Section 1.10) generalised from two boundary states to 2^n , and the standard tool for domino and polyomino tiling counts.

1.20 Counting Numbers

Problem Statement. Count the integers in the range $[a, b]$ that do not contain two adjacent equal digits (e.g. 33 and 12122 are excluded). Here $0 \leq a \leq b \leq 10^{18}$.

Pattern Recognition

“Count numbers up to N with a digit property” is **digit DP**. Reduce $[a, b]$ to $f(b) - f(a - 1)$ where $f(x)$ counts valid numbers in $[0, x]$, then build x ’s digits left to right, tracking just enough state to enforce “no two adjacent equal digits.”

State for digit DP

Write x as a digit string and fill positions left to right. The state is:

- **position** in the digit string,
- **previous digit** placed (to forbid an equal neighbour),
- **tight**: whether the prefix so far equals x ’s prefix (if so, the next digit is capped by x ’s digit),
- **started**: whether we have placed a nonzero digit yet (to handle leading zeros, which must not count as “adjacent equal”).

At each position we try every allowed next digit d (up to the cap if tight), skipping d equal to the previous digit once the number has started. Memoise over (position, previous, started) for the non-tight states.

Tight and Started Are the Two Universal Digit-DP Flags

Tight keeps the enumeration within $[0, x]$; it is true only along the single prefix that hugs x . **Started** distinguishes real leading digits from padding zeros, so a leading zero does not falsely collide with the next digit. Almost every digit-DP carries these two.

Algorithm

1. Define $f(x)$ = count of valid numbers in $[0, x]$ by digit DP over x 's digits with state (pos, prev, tight, started).
2. At each position try digits $0 \dots (\text{cap})$; skip $d = \text{prev}$ when started; recurse with updated flags.
3. Answer $f(b) - f(a - 1)$ (with $f(-1) = 0$ when $a = 0$).

Dry run

Dry Run. Count valid numbers in $[0, 23]$.

- Invalid two-digit numbers ≤ 23 with equal adjacent digits: 11, 22.
- All 24 numbers $0 \dots 23$ minus the 2 invalid ones = 22 valid numbers.
- The digit DP arrives at the same 22 by never placing a digit equal to its left neighbour.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string s;
5 long long memo[20][11][2];          // [pos][prev+1][started], for tight=
   false
6
7 long long solve(int pos, int prev, bool tight, bool started) {
8     if (pos == (int)s.size()) return 1;          // one valid number
   formed
9     if (!tight && memo[pos][prev + 1][started] != -1)
10         return memo[pos][prev + 1][started];
11
12     int cap = tight ? (s[pos] - '0') : 9;
13     long long total = 0;
14     for (int d = 0; d <= cap; d++) {
15         if (started && d == prev) continue;      // no equal adjacent
   digits
16         bool nstarted = started || (d > 0);
17         int nprev = nstarted ? d : -1;          // leading zeros carry
   prev=-1
18         total += solve(pos + 1, nprev, tight && (d == cap), nstarted);
19     }
20     if (!tight) memo[pos][prev + 1][started] = total;
21     return total;
22 }
23
24 long long f(long long x) {

```

```
25     if (x < 0) return 0;
26     s = to_string(x);
27     memset(memo, -1, sizeof(memo));
28     return solve(0, -1, true, false);
29 }
30
31 int main() {
32     long long a, b;
33     cin >> a >> b;
34     cout << f(b) - f(a - 1) << "\n";
35     return 0;
36 }
```

Complexity Analysis

Time Complexity. $O(\text{digits} \times 10 \times 10)$ per call – about $20 \cdot 10 \cdot 10$ states, each trying 10 digits: constant for 64-bit inputs. **Space Complexity.** $O(\text{digits} \times 10 \times 2)$ for the memo.

Key Takeaway

Digit DP counts numbers with a property by building them digit by digit under the two universal flags *tight* and *started*, memoising the free (non-tight) states. Range queries become $f(b) - f(a - 1)$. This template adapts to any per-digit constraint by changing the “skip this digit” rule.